

Object Oriented Analysis and Design

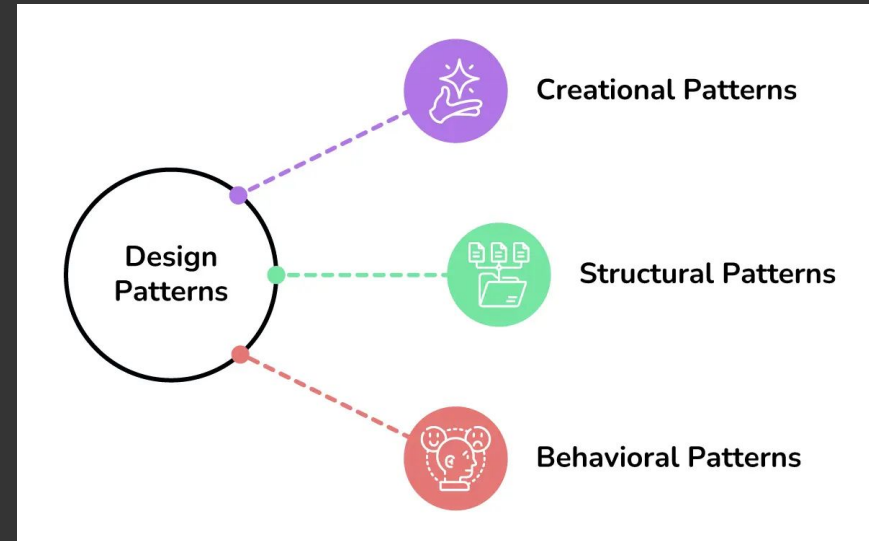
Design Patterns: Factory and Composite

Vladimir Soroka



3 Categories Of Patterns

- **Creational** – object creation
- **Structural** – class/object composition
- **Behavioral** – object interaction & responsibility



SHENKAR — **שנקר**
הנדסות, עיצוב, אמנות, ENGINEERING, DESIGN, ART

- # The Factory Design Pattern



The Factory Method Pattern

- **Concept**

- Delegates the instantiation logic to subclasses.
- Allows a class to defer instantiation to its children.



Factory Method UML

@startuml

title Factory Method Pattern - Document Creation System

' Abstract Product

interface Document {

+open()

+save()

}

' Concrete Products

class TextDocument implements Document {

+open()

+save()

}

class SpreadsheetDocument
implements Document {

+open()

+save()

}

class PresentationDocument
implements Document {

+open()

+save()

}

' Creator

abstract class
DocumentCreator {

+createDocument() :
Document

+openDocument()

}

' Concrete Creators

class TextDocumentCreator
extends DocumentCreator {

+createDocument() :
Document

}

class
SpreadsheetDocumentCreator
extends DocumentCreator {

+createDocument() :
Document

}

class
PresentationDocumentCreator
extends DocumentCreator {

+createDocument() :
Document

}

' Relationships

DocumentCreator --> Document :
<<creates>>

TextDocumentCreator -->
TextDocument : <<creates>>

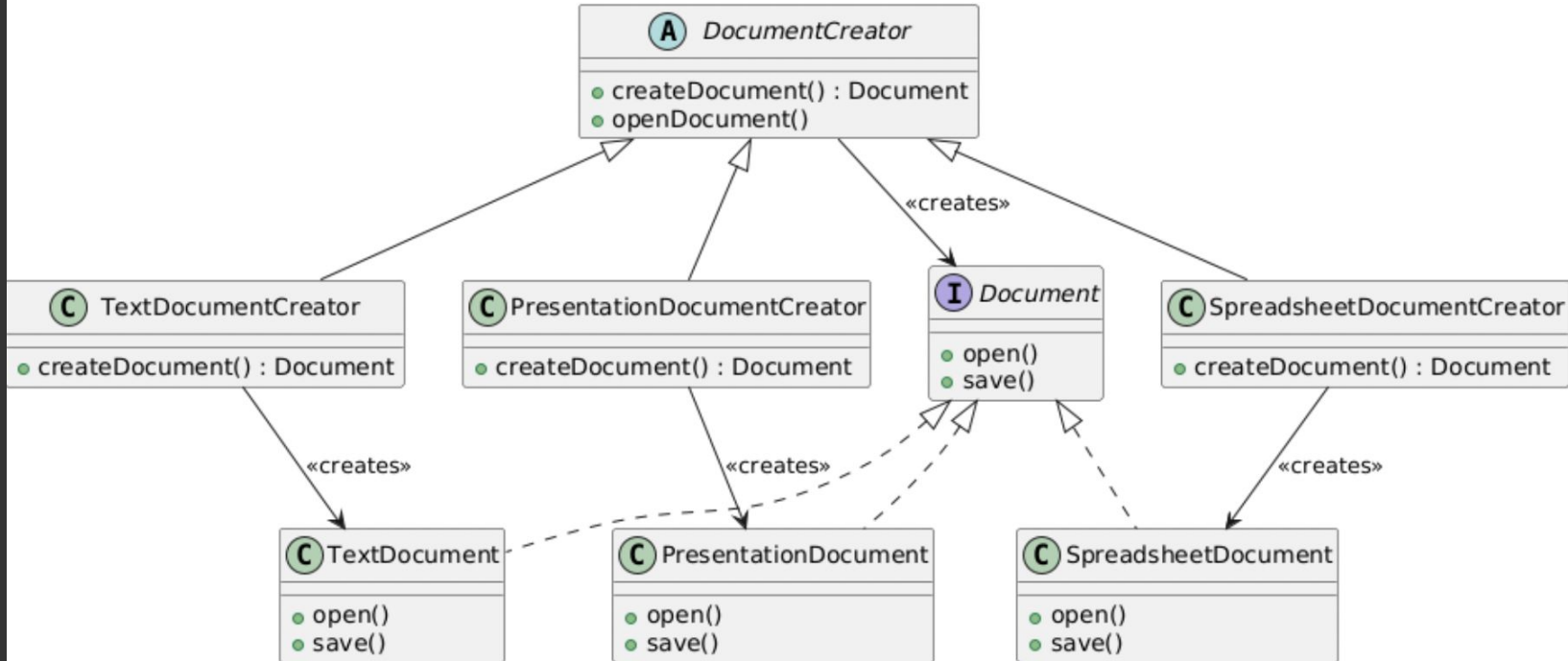
SpreadsheetDocumentCreator
--> SpreadsheetDocument :
<<creates>>

PresentationDocumentCreator
--> PresentationDocument :
<<creates>>

@enduml

Factory Method UML

Factory Method Pattern - Document Creation System



Factory Method Code

```
class Product {
public:
    virtual std::string getType() = 0;
};

class ConcreteProductA : public Product {
public:
    std::string getType() override { return "Product A"; }
};

class Creator {
public:
    virtual Product* factoryMethod() = 0;
};

class ConcreteCreatorA : public Creator {
public:
    Product* factoryMethod() override {
        return new ConcreteProductA();
    }
};
```

Factory Method: Is it A Pattern?

- **Argument: Factory Method is “just polymorphism”**
 - Critics argue that Factory Method is simply standard OO technique:
 - You create a virtual function and override it in a subclass to decide which object to instantiate.
 - This doesn’t introduce new behavior beyond basic inheritance.
- **Counter-Argument: Factory Method solves a recurring design problem**
 - Design patterns aren’t about novelty — they’re about codifying best practices.
 - Factory Method formalizes this idea:
 - It solves the problem of object creation without binding the code to concrete classes.
 - It enables loose coupling and extensibility.



Factory Method: Is it A Pattern?



```
class CarFactory {  
public:  
    Car* make() { return new Car(); }  
};
```

✓ It is a design pattern when used to **defer instantiation** to subclasses in a polymorphic way.

✗ It's **not really a pattern** if it's just a **named function that returns a new object**.

The Abstract Factory Pattern

- **Motivation**

- You need to create multiple related objects (like UI controls), and want to ensure they are consistent (Mac-style or Win-style).

- **Key Point**

- Group product creation logic together in a factory interface.



AF UML

@startuml

title Abstract Factory Pattern - Cross-Platform GUI System

left to right direction

' Abstract Products

interface Button {

+render()

}

interface Checkbox {

+render()

}

interface Slider {

+render()

}

' Concrete Products (Windows)

class WinButton implements Button {

+render()

}

class WinCheckbox implements Checkbox {

+render()

}

class WinSlider implements Slider {

+render()

}

' Concrete Products (Mac)

class MacButton implements Button {

+render()

}

class MacCheckbox implements Checkbox {

+render()

}

class MacSlider implements Slider {

+render()

}

' Abstract Factory

interface GUIFactory {

+createButton() : Button

}

+createCheckbox() : Checkbox

+createSlider() : Slider

}

' Concrete Factories

class WindowsGUIFactory implements GUIFactory {

+createButton() : Button

+createCheckbox() : Checkbox

+createSlider() : Slider

}

class MacGUIFactory implements GUIFactory {

+createButton() : Button

+createCheckbox() : Checkbox

+createSlider() : Slider

}

' Client

class Application {

-factory : GUIFactory

+Application(GUIFactory)

+createUI()

}

' Relationships

Application --> GUIFactory : uses

GUIFactory --> Button : <<creates>>

GUIFactory --> Checkbox : <<creates>>

GUIFactory --> Slider : <<creates>>

WindowsGUIFactory --> WinButton

WindowsGUIFactory --> WinCheckbox

WindowsGUIFactory --> WinSlider

MacGUIFactory --> MacButton

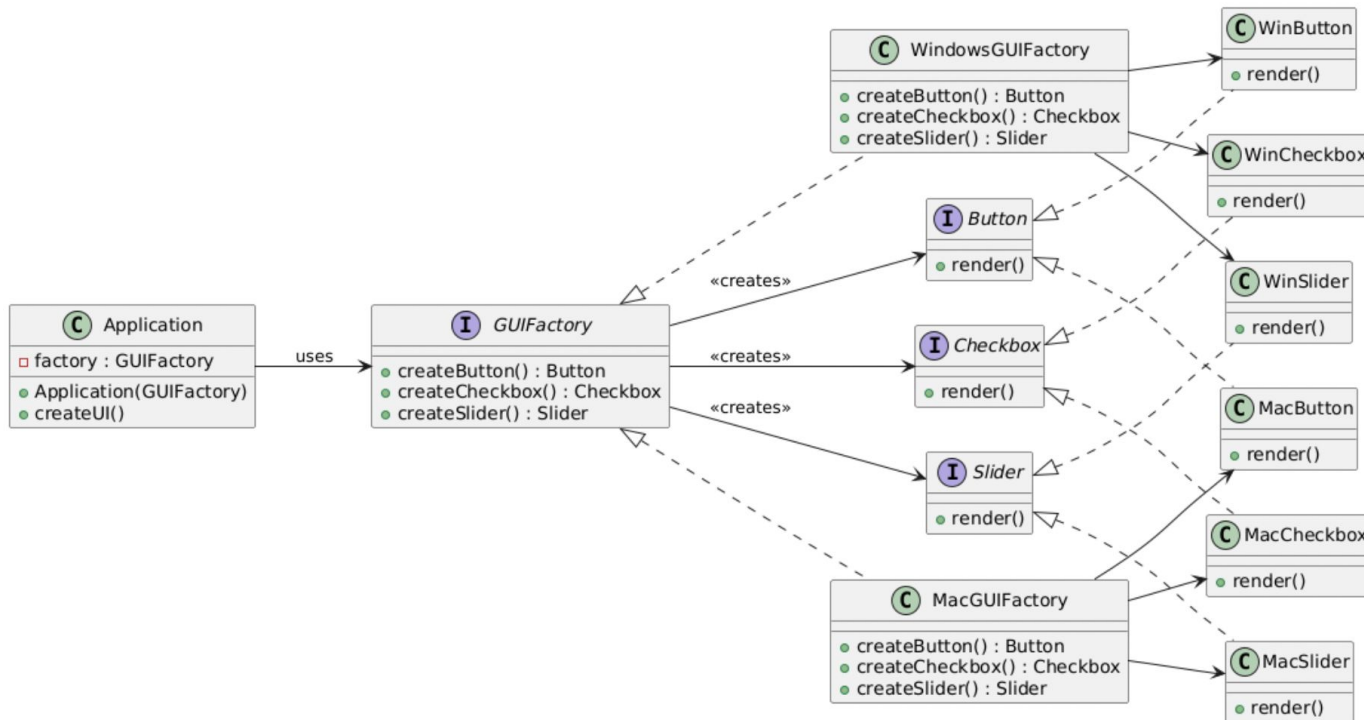
MacGUIFactory --> MacCheckbox

MacGUIFactory --> MacSlider

@enduml

AF UML

Abstract Factory Pattern - Cross-Platform GUI System



The AF Code

```
class Button {
public:
    virtual void render() = 0;
};

class MacButton : public Button {
public:
    void render() override { std::cout << "Mac Button\n"; }
};

class GUIFactory {
public:
    virtual Button* createButton() = 0;
};

class MacFactory : public GUIFactory {
public:
    Button* createButton() override { return new MacButton(); }
};
```

AF: Problems - Factory Explosion

Problem:

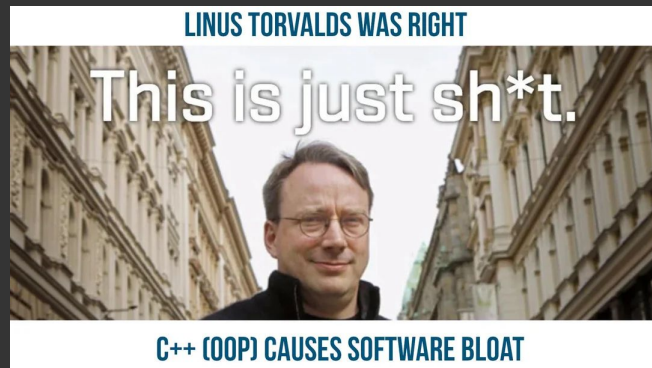
Each new product family or variant usually requires a **new factory class** and **new concrete product classes**.

Example:

- You have GUIFactory, MacGUIFactory, WindowsGUIFactory, LinuxGUIFactory
Each with createButton(), createCheckbox(), etc.
- Now add one new widget — Slider — and you must:
 - Update every interface
 - Add new MacSlider, WinSlider, LinuxSlider

Consequence:

- Violates **Open/Closed Principle** (OCP) — you must **modify** existing code to add a new product type.
- Scalability suffers



AF: Problems - Rigid Product Families

Problem:

The pattern enforces **families** of related objects — but what if your UI needs a **Mac Button** with a **Windows Checkbox**?

Consequence:

- You can't mix and match products from different factories.
- The pattern assumes strict consistency, which may be too restrictive.



AF: Problems - Over-Engineering

Problem:

Beginners often **overuse** Abstract Factory in systems where a **simple factory** or **direct instantiation** would suffice.

Symptoms:

- Too many interfaces
- Confusing indirection
- Harder debugging for what is just a new Button()

Quote (from Robert C. Martin):

“Design patterns are not about adding layers of indirection, they’re about solving problems. Don’t add machinery unless it solves a real problem.”



Bad Example

```
// Abstract Products
class Notification {
public:
    virtual void send() = 0;
};

class Logger {
public:
    virtual void logDelivery() = 0;
};

// Concrete Products
class EmailNotification : public Notification {
public:
    void send() override { std::cout << "Sending Email\n"; }
};

class SMSNotification : public Notification {
public:
    void send() override { std::cout << "Sending SMS\n"; }
};
```

AF: Problems - ISP Violation

Problem:

Abstract factories group all related creation methods into one interface:

```
class GUIFactory {  
    Button* createButton();  
    Checkbox* createCheckbox();  
    Slider* createSlider();  
}
```

But not all clients need all products.

Consequence:

Clients are **forced to depend** on creation methods they don't use — violating ISP.

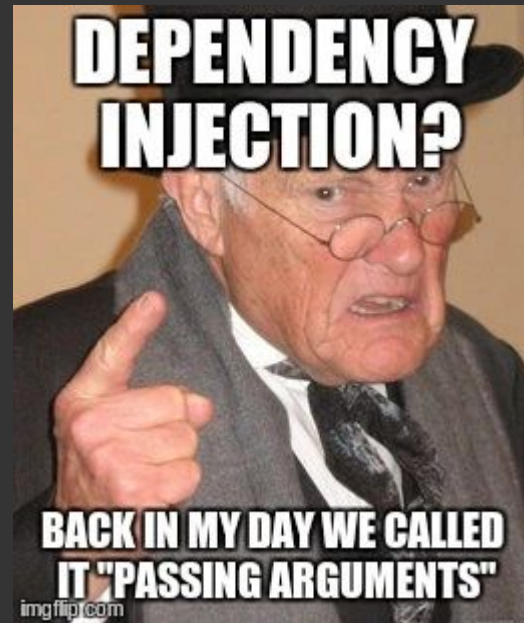
AF: Problems - Dependency Inversion?

Problem:

Abstract Factory is often used to inject dependencies manually, but **modern Dependency Injection (DI) frameworks** or **Service Locators** can do it more cleanly.

Alternative:

- Use a DI container that binds interfaces to concrete implementations at runtime.
- Abstract Factory becomes unnecessary.



AF: Problems - Inversion of Control Is Implicit

Problem:

The factory encapsulates object creation, but this may **hide important decisions** or **move configuration away from the developer**.

Result:

- Difficult debugging
- Hidden logic about what is being created



AF Problems: Summary

Summary: Pros vs Cons

✓ Strengths

Decouples creation from usage

Ensures consistency across product families

Easier testing / mocking of product variants

Supports substitution of entire product sets

✗ Problems / Controversies

Class explosion as products/families grow

Cannot mix product families

May overcomplicate small systems

Forces unwanted dependencies in client code

Abstract Factory - Summary

✓ Use it when:

- You truly need to create **families of related objects**
- The families must be **interchangeable** at runtime
- You plan for **UI theming, multi-platform systems,** or **plugin architectures**

✗ Avoid it when:

- You just need to create one or two kinds of objects
- You're not anticipating changes in the product family
- You can use **Dependency Injection** or **Builder** instead



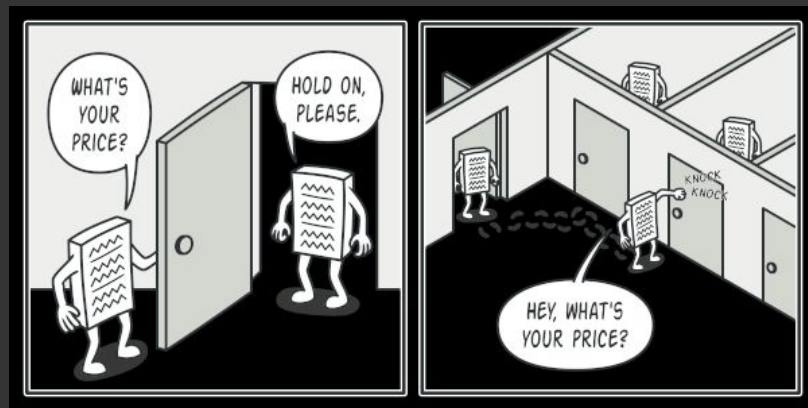
The Composite Pattern

- **Motivation**

- You want to treat individual and groups of objects uniformly.

- **Use Case**

- File system, UI layout, organization chart



Composite UML

```
@startuml
    title Composite Pattern - File System Hierarchy
    left to right direction

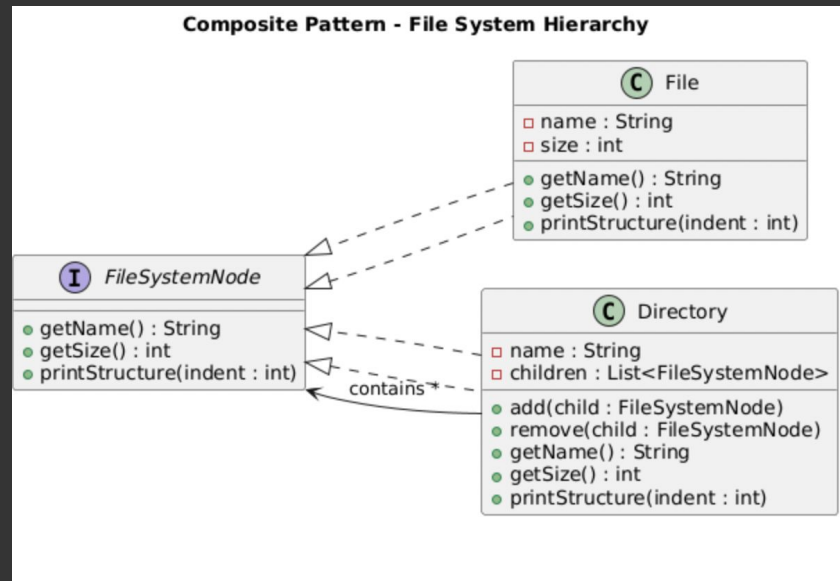
    ' Component
    interface FileSystemNode {
        +getName() : String
        +getSize() : int
        +printStructure(indent : int)
    }

    ' Leaf
    class File implements FileSystemNode {
        -name : String
        -size : int
        +add(child : FileSystemNode)
        +remove(child : FileSystemNode)
        +getName() : String
        +getSize() : int
        +printStructure(indent : int)
    }

    ' Composite
    class Directory implements FileSystemNode {
        -name : String
        -children : List<FileSystemNode>
        +add(child : FileSystemNode)
        +remove(child : FileSystemNode)
        +getName() : String
        +getSize() : int
        +printStructure(indent : int)
    }

    Directory --> FileSystemNode : contains *
    File ..|> FileSystemNode
    Directory ..|> FileSystemNode

    @enduml
```



The Composite Code

```
class Graphic {
public:
    virtual void draw() = 0;
};

class Line : public Graphic {
public:
    void draw() override { std::cout << "Line\n"; }
};

class Picture : public Graphic {
    std::vector<Graphic*> children;
public:
    void add(Graphic* g) { children.push_back(g); }
    void draw() override {
        for (auto g : children) g->draw();
    }
};
```

Composite: Problems - ISP Violation

Problem: The base component interface (e.g., Component or FileSystemNode) defines operations like add(), remove(), and getChild(), which only make sense for **composite nodes**, not leaves.

Why It's a Problem:

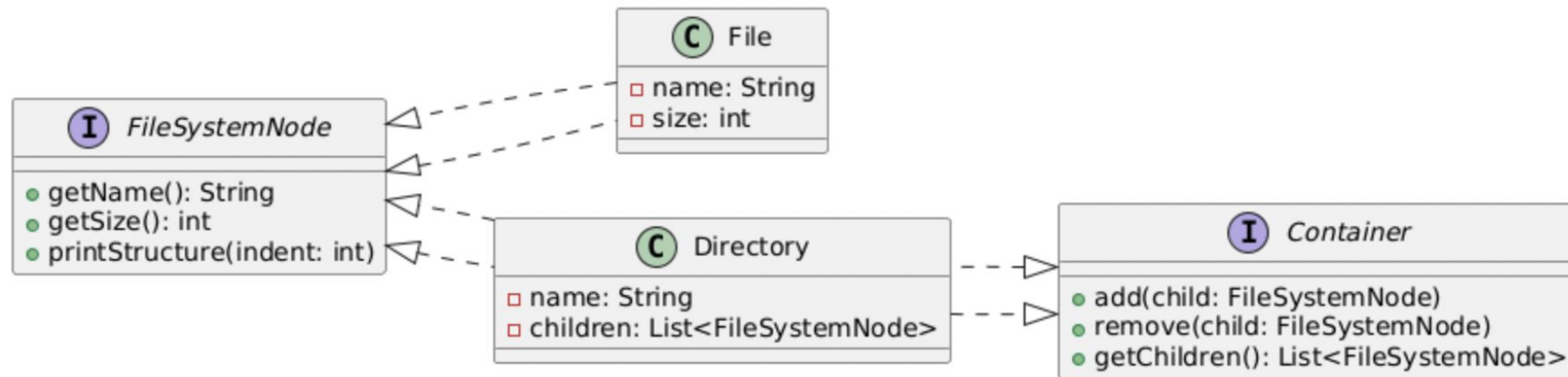
- **Leaf classes** are forced to implement (and often throw from) methods they don't need.
- This **violates ISP**, which says: *Clients should not be forced to depend on methods they do not use.*

Alternative:

- Use **separate interfaces** for leaves and composites, at the cost of losing uniform treatment.

```
class File : public FileSystemNode {  
    void add(FileSystemNode*) override {  
        throw std::runtime_error("Files cannot contain children");  
    }  
};
```

Fixed Example



Composite: Problems - Type Safety

Problem:

One of Composite's strengths — treating everything uniformly — can also be a downside.

Example:

A client may expect only File objects in a certain directory, but accidentally operates on Directory objects because everything is FileSystemNode.

Result:

- You lose **type safety** and may need **runtime type checks** (dynamic_cast, instanceof, etc.), which is considered a code smell in OOP.



Example

```
class Node {
public:
    virtual void add(Node* n) = 0;
    virtual void remove(Node* n) = 0;
    virtual int getSize() = 0;
};

class File : public Node {
public:
    void add(Node* n) override {
        throw std::runtime_error("Cannot add to file!");
    }
    void remove(Node* n) override {
        throw std::runtime_error("Cannot remove from file!");
    }
    int getSize() override { return size; }
};

class Directory : public Node {
public:
    void add(Node* n) override { children.push_back(n); }
    void remove(Node* n) override { /* ... */ }
    int getSize() override {
        int total = 0;
        for (Node* n : children)
            total += n->getSize();
        return total;
    }
};
```

```
if (dynamic_cast<Directory*>(node)) {
    node->add(new File("abc"));
}
```

Composite: Problems - Overuse on Trees

Problem:

Some developers **force Composite** onto every hierarchical structure — even when it's overkill or inflexible.

Example: Trying to model organizational hierarchies, HTML trees, or JSON using Composite too early.

Consequence:

- Tight coupling of structure and behavior
- Less flexible for operations like filtering, querying, or navigation
- Sometimes better to separate **structure** from **logic** (e.g., use the **Visitor** pattern or data-first design)

Learning about trees

botanics
students



computer science
students



Org Chart Composite Misuse

```
class Employee {  
public:  
    virtual void add(Employee* e) = 0;  
    virtual void remove(Employee* e) = 0;  
    virtual void displayHierarchy() = 0;  
    virtual double getCompensation() = 0;  
};
```

Org Chart Composite Misuse

```
class Engineer : public Employee {  
public:  
    void add(Employee* e) override {  
        throw std::runtime_error("Engineer cannot add subordinates!");  
    }  
    void remove(Employee* e) override {  
        throw std::runtime_error("Engineer cannot remove subordinates!");  
    }  
    void displayHierarchy() override { std::cout << "Engineer\n"; }  
    double getCompensation() override { return 80000.0; }  
};
```


Org Chart Composite Misuse

```
class Manager : public Employee {  
    std::vector<Employee*> subordinates;  
public:  
    void add(Employee* e) override { subordinates.push_back(e); }  
    void remove(Employee* e) override { /* remove from vector */ }  
    void displayHierarchy() override {  
        std::cout << "Manager\n";  
        for (auto e : subordinates) e->displayHierarchy();  
    }  
    double getCompensation() override {  
        double total = 100000.0;  
        for (auto e : subordinates) total += e->getCompensation();  
        return total;  
    }  
};
```

Composite Problems: Recursion

Problem:

Composite systems are often **recursive** and involve many objects (think: UI trees, DOM trees, file systems).

Issues:

- Memory usage from object trees
 - Cost of recursive traversal
- Event bubbling and propagation may become complex



Example

```
class Billable {
public:
    virtual double getCost() = 0;
    virtual void add(Billable* item) = 0;
};

class Task : public Billable {
    double cost;
public:
    Task(double c) : cost(c) {}
    double getCost() override { return cost; }

    void add(Billable* item) override {
        throw std::runtime_error("Cannot add to a Task");
    }
};

class Project : public Billable {
    std::vector<Billable*> items;
public:
    void add(Billable* item) override {
        items.push_back(item);
    }

    double getCost() override {
        double total = 0;
        for (auto i : items) {
            total += i->getCost(); // recursion here
        }
        return total;
    }
};
```

Composite Problems: Summary

✓ Strengths

Uniform access to hierarchies

Recursively composed behavior

Simplifies tree traversal logic

Extensible with patterns like Visitor

✗ Controversies / Drawbacks

Leaf classes forced to implement irrelevant methods

Loss of type safety and runtime type checks

Overuse in data models where behavior isn't needed

Poor fit for operations that differ deeply by node type